Contents

List of Figures

List of Tables

Chapter 1

Introduction

Chapter 2

Matching Problems and Algorithms

2.1 Definitions

Let $G = (V, E)$ be an undirected graph with $|V| = n$ vertices and $|E| = m$ edges. We call G bipartite if we can partition its vertex set into two sets V_1, V_2 such that for each edge $u, v \in E$, $u \in V_1$ and $v \in V_2$. If every vertex of V_1 is adjacent to every vertex of V_2 and $|V_1| = n_1, |V_2| = n_2$, then we call G a complete bipartite graph and denote it by K_{n_1,n_2}. By a path, we mean a simple path, i.e., a path in which no vertex is repeated. The length of a path is the number of edges it contains.

From set theory, we recall that the symmetric difference of two sets A and B, denoted by $A \triangle B$, is the set of elements that belong to A or B but not both. Thus, $A \triangle B = (A\backslash B) \cup (B\backslash A) = (A \cup B)\backslash(A \cap B)$. Also, if we want to add an element x to set A, we will use the shorthand $A + x := A \cup \{x\}$, and if we to remove it $A - x := A\backslash\{x\}$.

We write $T(n) = O(f(n))$ if and only if there exist positive constants c and n_0 such that $T(n) \leqslant cf(n)$ for all $n \geqslant n_0$. Most of the time, $T(n)$ will denote a bound on the worst-case running time of an algorithm defined on the positive integers. Similarly, we say that $T(n) = \Omega(f(n))$ if and only if there exist positive constants c and n_0 such that $T(n) \geqslant cf(n)$ for all $n \geqslant n_0$. Finally, we say that $T(n) = \Theta(f(n))$ if and only if $T(n) = O(f(n))$ and $T(n) = \Omega(f(n))$.

A matching of a graph $G = (V, E)$ is a subset of E such that each vertex is incident to at most one edge of the subset. The cardinality of a matching is the number of edges it contains. A vertex is called matched if it is incident to an edge of M. Otherwise, it is called unmatched or free. A matching is called maximal if no edge can be added without violating the matching property. A matching is maximum if no other matching has more edges. Clearly, every maximum matching is maximal (the converse is not always true). The following two definitions are fundamental in the study of matchings.

Definition 2.1.1. *A path P in G with respect to a matching M is called alternating if its edges are alternatingly out of and in M.*

Definition 2.1.2. *A path P in G with respect to a matching M is called augmenting if it is alternating and its endpoints are free.*

It is straightforward to see that every augmenting path has odd length. We will use the term M-augmenting whenever we want to emphasize the matching M.

2.2 Maximum Bipartite Matching

In this section, we present two famous algorithms for finding a maximum bipartite matching and prove their correctness.

The first algorithm is derived from a theorem of Berge [Ber57], previously observed by Petersen [Pet91].

Theorem 2.2.1. *Let $G = (V, E)$ be a graph and let M be a matching in G. Then M is maximum if and only if there is no M-augmenting path in G.*

Proof. Suppose that M is maximum. For the sake of contradiction, assume that there exists an M-augmenting path P with u and v as its endpoints in G. Then we can see that the matching $M^{'} = M \triangle E(P)$ is a valid matching in G and satisfies $|M^{'}| = |M| + 1$. Indeed, if we remove all edges of P that belong to M and add the remaining ones to M, we add one more edge to M in total (because P has odd length). The new matching is valid because u and v were free, and they were not incident to any matching edge, and every internal vertex of P remains incident to exactly one edge. All vertices outside P are unaffected. As a result, the new matching $M^{'}$ is valid and its cardinality is greater than that of M. This is a contradiction.

Now suppose that G does not contain any M-augmenting path and that a matching $M^{'}$ exists in G such that $|M^{'}| > |M|$. Consider the graph $M_0 = M \triangle M^{'}$. Each vertex in M_0 may be matched in both matchings or in either one. So, every vertex has degree at most 2. Hence, every component of M_0 is either a path or a cycle of even length (otherwise, there would exist a vertex that is incident to two edges in the same matching). Because M_0 has more edges of $M^{'}$ than of M, this also means that there exists a path P with more edges of $M^{'}$ than of M, and because the edges are alternating, P is an M-augmenting path, which contradicts our assumption that G does not contain any M-augmenting path. □

Augmenting-Path Algorithm. Based on the theorem above, we can write an algorithm that will try to find an augmenting path, alternate its edges (this will increase the size of the current matching by one), and repeat the whole process until we can't find any augmenting path. $N(v)$ represents all the vertices u such that $\{v, u\} \in E(G)$.

Algorithm 2.1 Augmenting Path Algorithm

Require: Graph $G = (V, E)$, vertex v
Ensure: Array matched such that $matched[u] = v$ and $u \in V_2$ and $v \in V_1$ if and only if $\{u, v\} \in E(G)$ belongs to the matching

```
function DfsAugmentingPath(int v, bool visited[], int matched[])
    if visited[v] then
        return FALSE;
    end if
    visited[v] ←TRUE;
    for all u ∈ N(v) do
        if matched[u] = ∅ or DfsAugmentingPath(matched[u], visited, matched) then
            matched[u] ← v;
            return TRUE;
        end if
    end for
    return FALSE;
end function
```

Algorithm 2.1 is getting called from all vertices in the left part of bipartite graph (let's call it V_1). We will use an extra array called visited to track the vertices from V_1 that we have already visited across the path and we have to reset it to be false for all these vertices before the initial call. Array matched marks the matched vertices only from the right part of the bipartite graph (let's call it V_2). To get the edges of the matching, we simply take all the edges $\{v, matched[v]\}$ such that $matched[v]$ is not empty and store them to set *matching*.

Algorithm 2.2 Maximum Bipartite Matching in $O(nm)$

Require: Bipartite graph $G = (V_1 \cup V_2, E)$
Ensure: Array *matching* that corresponds to the edges of the maximum matching

```
function MainAugmentingPath(int v, bool visited[], set matching)
    for all v ∈ V2 do
        matched[v] ← ∅;
    end for
    for all v ∈ V1 do
        for all u ∈ V1 do
            dist[i] ← FALSE;
        end for
        DfsAugmentingPath(v, visited, matched);
    end for
    matching ← ∅;
    for all v ∈ V2 do
        if matched[v] ≠ ∅ then
            matching ← matching ∪ {{matched[v], v}};
        end if
    end for
    return matching;
end function
```

The idea is that if the initial call is from a free vertex, say v, and we cannot find another free vertex u such that the edge $\{v, u\}$ exists, then we simply try to find a path (v, x, y) such that $\{v, x\}$ is not in the matching, but $\{x, y\}$ is, so we can start trying again from the vertex y, until we find a free vertex and then recursively alternate the path.

Lemma 2.2.2. *If we call Algorithm 2.1 from a free vertex v and augmenting path with endpoint v exists, then it will return* TRUE *and alternate its edges.*

Proof. Suppose that a free vertex $u \in V_2$ exists such that $\{v, u\} \in E(G)$. Then from line 7 the algorithm successfully finds an augmenting path. Now suppose that such a vertex doesn't exist and $P = (v, v_1, v_2, ..., u)$ any augmenting path with endpoint the vertex v. Since P is alternating, the edge $\{v, v_1\}$ does not belong to the matching, while $\{v_1, v_2\}$ does. Because the algorithm checks all neighbors of v (line 6), it will eventually examine v_1 (assuming that all the other neighbors - if they exist - return false), and line 7 does a recursive call for $matched[v_1] = v_2$ (note that v_2 is impossible to be visited, otherwise an alternative path $(v, ..., v_2)$ exists and the algorithm would have already found another augmenting path from that recursive call to u or to another free vertex). Repeating the same argument along P, the algorithm eventually reaches the free vertex $u \in V_2$, where line 7 is true since $matched[u] = \varnothing$. The recursive calls then return TRUE and update the array matched accordingly, thus alternating the edges of P. □

From this, we get the following remark.

Remark 2.2.3. *After an initial call of Algorithm 2.1 from a free vertex $v \in V_1$, the only vertices that can become matched are v and one free vertex of V_2. If no augmenting path is found, then the matching remains unchanged.*

Then, we can prove the following lemma.

Lemma 2.2.4. *Every initial call to Algorithm 2.1 from Algorithm 2.2 takes a free vertex as parameter.*

Proof. This is obviously true in the first iteration. Assume that we are at iteration i, and that the statement holds for all previous iterations. Let v be the vertex from which we are about to initially call Algorithm 2.1. If v is free, then we are done. If it is not, v must have become matched during some previous iteration. However, by Remark 2.2.3, in each iteration the only vertex of V_1 that can become matched is the vertex from which Algorithm 2.1 was initially called. Since Algorithm 2.1 has never been initially called from v before iteration i, the vertex v could not have become matched earlier. □

The output of the Algorithm 2.2 (set matching) is always a valid matching. Since the algorithm initializes with an empty matching and only updates it by alternating edges along valid augmenting paths starting from free vertices (Lemma 2.2.2), the maintained set of edges is always a valid matching. Now we have to prove that it is maximum.

Theorem 2.2.5. *Algorithm 2.2 returns a maximum matching.*

Proof. Let $v_1, v_2, \ldots, v_k$ be the vertices of V_1 in the order in which Algorithm 2.2 processes them. For each $i \in \{0, 1, \ldots, k\}$, let M_i be the matching after the first i iterations of the algorithm, with $M_0 = \varnothing$. Also, let G_i be the subgraph of G induced by the vertex set

$$\{v_1, v_2, \ldots, v_i\} \cup V_2.$$

We will prove by induction on i that M_i is a maximum matching of G_i.

For $i = 0$, the graph G_0 has no vertices in V_1, so the empty matching M_0 is trivially maximum.

Now assume that M_i is a maximum matching of G_i, and consider iteration $i+1$, where the algorithm processes the vertex v_{i+1}. By Lemma 2.2.4, the initial call to Algorithm 2.1 is made from a free vertex. We consider two cases.

Case 1: *DfsAugmentingPath*(v_{i+1}) (Algorithm 2.1 with parameter the vertex v_{i+1}) returns TRUE. Then, by Lemma 2.2.2, the algorithm finds an augmenting path in G_{i+1} with endpoint v_{i+1} and alternates its edges. Therefore,

$$|M_{i+1}| = |M_i| + 1.$$

On the other hand, any matching of G_{i+1} can use at most one edge incident to the new vertex v_{i+1}. Hence, by removing that edge if it exists, we obtain a matching of G_i, and therefore every matching of G_{i+1} has size at most $|M_i| + 1$. Since M_i is maximum in G_i, it follows that M_{i+1} is a maximum matching of G_{i+1}.

Case 2: *DfsAugmentingPath*(v_{i+1}) returns FALSE. Then, again by Lemma 2.2.2, there is no M_i-augmenting path in G_{i+1} with endpoint v_{i+1}. Since M_i is maximum in G_i, Theorem 2.2.1 implies that there is no M_i-augmenting path contained entirely in G_i. Thus, there is no M_i-augmenting path in G_{i+1} at all, because any augmenting path in G_{i+1} that is not contained in G_i must involve the only new vertex of V_1, namely v_{i+1}. Therefore, by Theorem 2.2.1, M_i is already a maximum matching of G_{i+1}. Since the matching does not change in this case, we have

$$M_{i+1} = M_i,$$

and so M_{i+1} is a maximum matching of G_{i+1}.

In both cases, M_{i+1} is a maximum matching of G_{i+1}. This completes the induction. □

Before each search, we initialize the visited array which takes $O(n)$ time. We call *DfsAugmentingPath* for each vertex $v \in V_1$ searching for an augmenting path. To find an augmenting path, we will visit each vertex and each edge at most once (similar to the classic DFS), so the total complexity is $O(nm)$ when $m = \Omega(n)$.

Hopcroft-Karp Algorithm Hopcroft and Karp [HK73] provided a faster algorithm, again based on finding augmenting paths, but in a smarter way. The algorithm runs in phases. In each phase, we find a maximal set of vertex-disjoint shortest M-augmenting paths and take the symmetric difference of the current matching M with the union of these paths. The process stops when no augmenting path can be found.

Our implementation is based on the version of Shimon Even and Alon Itai, who made significant modifications to Dinic's algorithm for the maximum flow problem [Din70, Din06]. As we will see at the end of this section, maximum flow and maximum bipartite matching are closely connected problems.

The key idea of the algorithm is to efficiently find these paths and alternate their edges. To do that, we construct a new graph called the layered network. We describe this graph with an array called level, defined on the vertices of V_1. Initially, every free vertex of V_1 gets level 1. Then, if a vertex $u \in V_1$ has level i, every vertex of V_1 that can be reached from u by first taking an unmatched edge to a vertex of V_2 and then a matched edge back to V_1 gets level $i+1$, provided it has not already received a level. The vertices are visited in Breadth-First Search (BFS) order, starting from all free vertices of V_1. In this way, *level*$[v]$

represents the minimum alternating distance of $v \in V_1$ from the set of free vertices of V_1 in the current phase. If all edges in the layered network are directed from a source vertex in V_1 to a destination vertex in V_2, it is clear that the resulting graph is a dag (directed acyclic graph).

Algorithm 2.3 BFS Layered Network Construction

Require: Bipartite graph $G = (V_1 \cup V_2, E)$, arrays *next*, *prev* represent the current matching
Ensure: Array *level* for the vertices of V_1 describes the layered network

```
function BfsLayeredNetwork(int next[], int prev[], int level[])
    treeDepth ← 0;
    initialize empty queue Q;
    for all u ∈ V_1 do
        if next[u] = ∅ then
            level[u] ← 1;
            Q.push(u);
        end if
    end for
    while not Q.isEmpty() do
        u ← Q.pop();
        for all x ∈ N(u) do
            if prev[x] = ∅ then
                treeDepth ← level[u];
                continue;
            end if
            if prev[x] = u then
                continue;
            end if
            if level[prev[x]] = 0 then
                level[prev[x]] ← level[u] + 1;
                Q.push(prev[x]);
            end if
        end for
        if treeDepth ≠ 0 then
            break;
        end if
    end while
    for all u ∈ V_1 do
        if level[u] > treeDepth then
            level[u] ← 0;
        end if
    end for
    return (treeDepth ≠ 0);
end function
```

The first time we encounter a free vertex in V_2, we determine the last useful level in the current phase and we store it in the variable *treeDepth*. From that, we can also determine the length of a shortest augmenting path in the current phase (which is $2 \cdot \textit{treeDepth} - 1$). We do not need to continue the BFS deeper than that level and the vertices that remain in the queue belong either to the same level or to the next one. The vertices of the same level may still be used in the searching step to find other shortest augmenting paths, since we have already assigned their correct level. On the other hand, vertices that have already been assigned level $treeDepth + 1$ correspond to longer paths, so we reset their level to 0.

We define an array *next* for every $v \in V_1$ such that $next[v] = u$ if and only if the edge $\{v, u\}$ belongs to the current matching; otherwise, $next[v] = \varnothing$. Similarly, for every $u \in V_2$, we define $prev[u] = v$ if and only if the edge $\{u, v\}$ belongs to the current matching; otherwise, $prev[u] = \varnothing$. Thus, the arrays *next* and *prev* represent the current matching and are updated whenever we find an augmenting path and alternate its edges.

Theorem 2.2.6. *Algorithm 2.3 correctly computes the array level of the layered network of the current matching M. More precisely, if it returns* FALSE, *then no M-augmenting path exists in G. Otherwise, for every vertex $v \in V_1$, we have $level[v] \neq 0$ if and only if there exists an M-alternating path from a free vertex of V_1 to v. Moreover, if $level[v] \neq 0$, then the length of a shortest such path is $2 \cdot level[v] - 2$.*

Proof. For every free vertex $v \in V_1$, the algorithm sets $level[v] = 1$, and the corresponding alternating path has length 0. Now suppose that a vertex $u \in V_1$ is popped from the queue and that the statement already holds for u. Let $x \in V_2$ be a neighbor of u.

If $prev[x] = \varnothing$, then extending a shortest alternating path to u by the edge $\{u, x\}$ gives an M-augmenting path of length $2 \cdot level[u] - 1$. If $prev[x] = u$, then $\{u, x\}$ is a matched edge and cannot be used to continue an alternating path. Otherwise, if $prev[x] = w \neq u$ and $level[w] = 0$, then by appending the unmatched edge $\{u, x\}$ and the matched edge $\{x, w\}$ to a shortest alternating path ending at u, we obtain an alternating path from a free vertex of V_1 to w of length $(2 \cdot level[u] - 2) + 2 = 2 \cdot (level[u] + 1) - 2$. Hence, the algorithm correctly sets $level[w] = level[u] + 1$. Since vertices are processed in BFS order, this is the minimum possible alternating distance to w.

Therefore, for every vertex $v \in V_1$ with $level[v] \neq 0$, the value $level[v]$ is exactly the level of v in the layered network, and the length of a shortest alternating path from a free vertex of V_1 to v is $2 \cdot level[v] - 2$.

Finally, if the algorithm returns FALSE, then no free vertex of V_2 is reached during the BFS. Therefore, there is no M-augmenting path in G. Indeed, if such a path existed, then the vertex just before its free endpoint on a shortest such path would lie in the layered network, and the free vertex in V_2 would also be reached. □

Corollary 2.2.7. *If Algorithm 2.3 returns* TRUE, *then the length of a shortest M-augmenting path is $2 \cdot treeDepth - 1$.*

The searching step, Algorithm 2.4, is almost the same as Algorithm 2.1, so we will omit the proof. This time, we move across the layered network based on the *level* array. We also use an array *ptr* for all vertices in V_1. This array, holds for each vertex a counter that indicates the first neighbor that has not been checked yet. Hence, if we return to the same vertex in the same phase, we do not examine again the neighbors that have already returned FALSE. In particular, if $ptr[u]$ reaches the end of the adjacency list of u, then no augmenting path can be found through u in the current phase.

We could of course combine the two *if* statements into one.

Algorithm 2.4 DFS Phase of Hopcroft-Karp

Require: Bipartite graph $G = (V_1 \cup V_2, E)$, vertex $u \in V_1$, arrays *next*, *prev*, *level*, *ptr*
Ensure: Returns TRUE if a shortest augmenting path is found from u

function *DfsHopcroftKarp*(`int` u, `int` $next[]$, `int` $prev[]$, `int` $level[]$, `int` $ptr[]$)
    for $ptr[u]$ **to** $|N(u)| - 1$ **do**
        $x \leftarrow$ the $ptr[u]$-th neighbor of u;
        if $prev[x] = \varnothing$ **then**
            $prev[x] \leftarrow u$;
            $next[u] \leftarrow x$;
            $ptr[u] \leftarrow |N(u)|$;
            return TRUE;
        end if
        if $level[prev[x]] = level[u] + 1$ **and** *DfsHopcroftKarp*($prev[x]$, *next*, *prev*, *level*, *ptr*) **then**
            $prev[x] \leftarrow u$;
            $next[u] \leftarrow x$;
            $ptr[u] \leftarrow |N(u)|$;
            return TRUE;
        end if
    end for
    return FALSE;
end function

Algorithm 2.5 and lines 9-21 run the phases of the algorithm. In each phase, we call Algorithm 2.3 (line 9) to construct the layered network, find all augmenting paths of this graph by calling Algorithm 2.4 for each free vertex of V_1 (lines 13-17), and repeat the whole process. Note that we have to initialize the *ptr* array to 0 before starting the search, because all vertices will check their first neighbor first. We also have to initialize the *level* array to zero before building the layered network. The set *matching* contains the edges of the maximum matching.

Algorithm 2.5 Hopcroft–Karp Algorithm

Require: Bipartite graph $G = (V_1 \cup V_2, E)$
Ensure: Array *matching* contains the edges of the maximum matching

1: **function** *HopcroftKarp*( )
2:     **for all** $u \in V_1$ **do**
3:         $next[u] \leftarrow \varnothing$;
4:         $level[u] \leftarrow 0$;
5:     **end for**
6:     **for all** $x \in V_2$ **do**
7:         $prev[x] \leftarrow \varnothing$;
8:     **end for**
9:     **while** *BfsLayeredNetwork*(*next*, *prev*, *level*) **do**
10:         **for all** $u \in V_1$ **do**
11:             $ptr[u] \leftarrow 0$;
12:         **end for**
13:         **for all** $u \in V_1$ **do**
14:             **if** $next[u] = \varnothing$ **then**
15:                 *DfsHopcroftKarp*(u, *next*, *prev*, *level*, *ptr*);
16:             **end if**
17:         **end for**
18:         **for all** $u \in V_1$ **do**
19:             $level[u] \leftarrow 0$;
20:         **end for**
21:     **end while**
22:     $matching \leftarrow \varnothing$;
23:     **for all** $u \in V_1$ **do**
24:         **if** $next[u] \neq \varnothing$ **then**
25:             $matching \leftarrow matching \cup \{\{u, next[u]\}\}$;
26:         **end if**
27:     **end for**
28:     **return** *matching*;
29: **end function**

It is evident that constructing the layered network takes no more than $O(n+m)$. Moreover, lines 13-17 take no more than $O(n+m)$ as each edge is examined at most once (with the help of *ptr* array) and as a result when vertex returns FALSE, we are not going to process it again in the current phase.

Lemma 2.2.8. *Each phase, that is calculating the maximal set of vertex-disjoint shortest M-augmenting paths and take the symmetric difference with current matching M takes no more than $O(n+m)$ time.*

We shall show that we need at most $O(\sqrt{n})$ phases, but before that, we have to prove some important lemmas.

Lemma 2.2.9. *Let M and M' be two matchings of a graph G, with $|M'| > |M|$. Then the graph $M \triangle M'$ contains at least $|M'| - |M|$ vertex-disjoint M-augmenting paths.*

Proof. We follow the same logic as in Theorem 2.2.1. We consider the graph $M^* = M \triangle M'$. We can see that the graph contains components, each of which is an alternating path or an even-length cycle. Cycles have the same number of edges from M and M', and the same holds for alternating paths of even length. Each component that represents an augmenting path has one more edge from one matching than from the other. In fact, every M-augmenting path has one more edge from M' than M. From $|M'| > |M|$, we can see that we have more M-augmenting paths than M'-augmenting paths, and there must be at least $|M'| - |M|$ such paths. This completes the proof. ☐

The following is a special case of Lemma 2.2.9.

Lemma 2.2.10. *Given a matching M and a maximum matching M^*, the graph contains $|M^*| - |M|$ vertex-disjoint M-augmenting paths.*

Lemma 2.2.11. *Let $P = \{P_1, P_2, \cdots, P_t\}$ be a maximal set of vertex-disjoint shortest M-augmenting paths with length x obtained during a single phase of Algorithm 2.5 and $M' = M \triangle (P_1 \cup P_2 \cup \cdots \cup P_t) = M \triangle P_1 \triangle P_2 \triangle \cdots \triangle P_t$ with Q be an M'-augmenting path. The set $R = (P_1 \cup P_2 \cup \cdots \cup P_t) \triangle Q$ has at least $x(t+1)$ edges.*

Proof. From $M' = M \triangle (P_1 \cup P_2 \cup \cdots \cup P_t)$, we have

$$P_1 \cup P_2 \cup \cdots \cup P_t = M \triangle M'$$

That is,

$$R = (M \triangle M') \triangle Q = M \triangle (M' \triangle Q)$$

As Q is an M'-augmented path, $M' \triangle Q$ has size of $|M'| + 1 = |M| + t + 1$ (plus $t+1$ because it will get $t+1$ more edges from alternating the paths of P and the path Q). By Lemma 2.2.9, there is at least $|M' \triangle Q| - |M| = t + 1$ M-augmenting paths. Because every M-augmenting path has length at least x, it follows that R has at least $x(t+1)$ edges. ☐

Lemma 2.2.12. *The length of the shortest augmenting path increases in every phase.*

Proof. Let's assume that the current matching is M and denote by $P = \{P_1, P_2, \cdots, P_t\}$ the maximal set of vertex-disjoint shortest M-augmenting paths of length x in one phase of Algorithm 2.5. Moreover, suppose that we take $M' = M \triangle (P_1 \cup P_2 \cup \cdots \cup P_t)$ (the matching that we get at the end of phase) and let Q be an M'-augmenting path. Obviously, Q has a common vertex v with at least one path from P, otherwise, it would mean that Q is also in M and either $|Q| > x$ (in this case we are done) or it would contradict the maximality of the set. Let $R = (P_1 \cup P_2 \cup \cdots \cup P_t) \triangle Q$. From Lemma 2.2.11, R has at least $x(t+1)$ edges. Let $P_i \in P$ such that $v \in P_i$. We observe that v is incident with a matched edge in M' because at the end of the phase we alternate the edges of each path in P. The endpoints of Q do not belong to M' because they are free, so it is guaranteed that v is an internal vertex of Q and therefore it is also incident with a matched edge in M'. Obviously, it's impossible for v to be incident with two different matched edges from M', so P_i and Q have at least one edge from M' in common. It follows that

$$xt + x \leqslant |R| \leqslant |P_1 \cup P_2 \cup \cdots \cup P_t \cup Q| \leqslant xt + |Q| - 1$$

and hence $x + 1 \leqslant |Q|$. ☐

Lemma 2.2.13. *Algorithm 2.5 performs at most $O(\sqrt{n})$ phases.*

Proof. Assume that we have already performed $\sqrt{n}$ phases and the current matching is M. From Lemma 2.2.10, we know that there exist exactly $|M^*| - |M|$ vertex-disjoint M-augmenting paths for a maximum matching M^*. From Lemma 2.2.12 all these augmenting paths will have length at least $\sqrt{n}$, so there cannot be more than $\sqrt{n}$ (we would need more than n vertices) of them, which means that, $|M^*| - |M| \leqslant \sqrt{n}$. It follows from this that we can't perform more than $\sqrt{n}$ phases from that point. We conclude that the total phases can't be more than $2\sqrt{n} = O(\sqrt{n})$. □

From Lemma 2.2.8 and Lemma 2.2.13 we get the complexity of Hopcroft-Karp algorithm.

Theorem 2.2.14. *Algorithm 2.5 returns a maximum matching in $O((n+m)\sqrt{n})$ time.*

Maximum Bipartite Matching from Max-Flows We can reduce the bipartite maximum matching problem to the maximum flow problem by adding two extra nodes to the initial graph. The first vertex is called the source and will point to all vertices in V_1 with directed edges, and the second vertex is called the sink, with all vertices from V_2 pointing to it. Moreover, all the edges of the initial graph will turn into directed edges, pointing to V_2 from a vertex of V_1. Assuming capacities of one for each edge (so there exists an integral maximum flow, that is every edge will have flow of 0 or 1), the maximum flow from source to sink is equal to the maximum bipartite matching. Actually, it is easy to see this because all paths that carry flow from source to sink should be edge-disjoint, and each path will have exactly one edge from V_1 to V_2, so no other path can use this edge again. We can get a maximum flow in the resulting graph from a maximum matching in the initial graph by just taking all the edges of matching and picking the edge from source to the V_1 vertex, and the edge from V_2 to the sink.

Consequently, someone could make the above reduction and use a famous flow algorithm, like Dinic [Din70] to solve the maximum bipartite problem with $O(n^2m)$. It turns out, that using some kind of the arguments we talked before, that Dinic's algorithm will perform with the same complexity as Hopcroft-Karp algorithm.

Recent Research Maximum flow is one of the most well-known problems in computer science and has been studied extensively over the years. Finding faster algorithms for the maximum flow problem, and and especially for the minimum-cost flow problem (to which maximum flow can be reduced), could lead to breakthroughs in many areas. For a long time, algorithms based primarily on combinatorial methods were far from the almost-linear-time bounds achieved by more recent continuous techniques. In 2023, a deterministic $m^{1+o(1)}$-time algorithm for exact maximum flow and minimum-cost flow was discovered [BCK+23]. This line of work also implies an $m^{1+o(1)}$-time algorithm for the maximum bipartite matching problem. On the other hand, randomized combinatorial algorithms were later obtained independently for exact maximum flow [BBST24] and for maximum bipartite matching [CK24].

2.3 The Stable Marriage Problem

Assume that we are given a set of n men $\{m_1, m_2, \ldots, m_n\}$ and n women $\{w_1, w_2, \ldots, w_n\}$, where each man has a strict preference ranking over all n women, and each woman also has a strict preference ranking over all n men. We say that a *matching* is a set of unordered pairs consisting of a man and a woman (similarly to the previous section, where we considered the undirected edges of a bipartite graph) and each of them appears at most once. A *marriage* is a perfect matching between men and women, meaning that every man and every woman is matched exactly once. A marriage is *unstable* if there exists a pair $\{m_i, w_j\}$ that does not belong to the matching such that m_i prefers w_j over his actual partner, and w_j prefers m_i over her actual partner. We call such a pair a *blocking pair*. Otherwise, we say that it is *stable*.

The following example shows a stable marriage.

Example 2.3.1. *Consider an instance of four men*

$$\{m_1, m_2, m_3, m_4\}$$

and four women

$$\{w_1, w_2, w_3, w_4\}.$$

The preference lists of the men are given by

man	1^{st}	2^{nd}	3^{rd}	4^{th}
m_1	w_1	w_2	w_3	w_4
m_2	w_2	w_1	w_3	w_4
m_3	w_3	w_4	w_1	w_2
m_4	w_4	w_3	w_1	w_2

and the preference lists of the women are

woman	1^{st}	2^{nd}	3^{rd}	4^{th}
w_1	m_2	m_1	m_3	m_4
w_2	m_1	m_2	m_3	m_4
w_3	m_4	m_3	m_1	m_2
w_4	m_3	m_4	m_1	m_2

Now consider the matching

$$M = \{\{m_1, w_2\}, \{m_2, w_1\}, \{m_3, w_4\}, \{m_4, w_3\}\}.$$

We can see that M is stable. Indeed, each woman is matched to her first choice:

w_1 *is matched with* m_2, $\quad w_2$ *is matched with* m_1, $\quad w_3$ *is matched with* m_4, $\quad w_4$ *is matched with* m_3.

Hence, no woman prefers any other man to her assigned partner. Therefore, no blocking pair can exist, since a blocking pair would require both agents to prefer each other over their current partners. Consequently, M is a stable matching. Moreover, this stable matching is not unique, since

$$M' = \{\{m_1, w_1\}, \{m_2, w_2\}, \{m_3, w_3\}, \{m_4, w_4\}\}$$

is also stable.

Example 2.3.2. *Consider an instance of four men*

$$\{m_1, m_2, m_3, m_4\}$$

and four women

$$\{w_1, w_2, w_3, w_4\}.$$

The preference lists of the men are given by

man	1^{st}	2^{nd}	3^{rd}	4^{th}
m_1	w_1	w_2	w_3	w_4
m_2	w_2	w_3	w_1	w_4
m_3	w_2	w_1	w_4	w_3
m_4	w_3	w_4	w_1	w_2

and the preference lists of the women are

woman	1^{st}	2^{nd}	3^{rd}	4^{th}
w_1	m_3	m_1	m_2	m_4
w_2	m_1	m_2	m_3	m_4
w_3	m_2	m_4	m_1	m_3
w_4	m_4	m_3	m_1	m_2

Now consider the matching

$$M = \{\{m_1, w_1\}, \{m_2, w_2\}, \{m_3, w_4\}, \{m_4, w_3\}\}.$$

We claim that M is not stable. Indeed, the pair $\{m_3, w_1\}$ is a blocking pair, since m_3 prefers w_1 to w_4, while w_1 prefers m_3 to m_1. Thus, both m_3 and w_1 prefer each other to their assigned partners under M. Hence, M admits a blocking pair and therefore it is not a stable matching.

Gale and Shapley [GS62] provided an algorithm that always outputs a stable marriage. The algorithm is quite simple. In each iteration of the algorithm, we try to match a man who is not matched to any woman by proposing to the woman he currently prefers most, and the woman accepts the best proposal she receives. If a man is matched with a woman and she later receives a better proposal, then she will leave her current partner and accept that proposal. The man will become unmatched again, and we will try to match him again in a later iteration, starting from the next woman in his preference list.

Algorithm 2.6 Gale–Shapley Algorithm

Require: Preference arrays *men*, *women* and their size n

Ensure: Array *manPartner* contains a stable matching

```
1: function StableMarriage(int n, int men[][], int women[][])
2:     initialize empty queue Q;
3:     for i ← 0 to n − 1 do
4:         ptr[i] ← 0;
5:         womanPartner[i] ← ∅;
6:         manPartner[i] ← ∅;
7:         Q.push(i);
8:     end for
9:     for i ← 0 to n − 1 do
10:        for j ← 0 to n − 1 do
11:            rank[i][women[i][j]] ← j;
12:        end for
13:    end for
14:    while not Q.isEmpty() do
15:        m ← Q.pop();
16:        while manPartner[m] = ∅ do
17:            w ← men[m][ptr[m]];
18:            ptr[m] + +;
19:            if womanPartner[w] = ∅ then
20:                womanPartner[w] ← m;
21:                manPartner[m] ← w;
22:            else
23:                if rank[w][m] < rank[w][womanPartner[w]] then
24:                    manPartner[m] ← w;
25:                    Q.push(womanPartner[w]);
26:                    manPartner[womanPartner[w]] ← ∅;
27:                    womanPartner[w] ← m;
28:                end if
29:            end if
30:        end while
31:    end while
32:    return manPartner;
33: end function
```

The input of the algorithm is the size n and the preference lists *women* and *men*. *women*$[i]$ is the preference list of woman i and *men*$[j]$ is the preference list of man j. Arrays *womanPartner* and *manPartner* represent the current matching. We will use two extra arrays that will help us see if a proposal can be made and whether it benefits us. Array *ptr* will store, for a man i, the most highly preferred woman that he has not proposed to yet. With this array, we will be able to know the answer immediately. Array *rank* will help us determine, for a woman, whether she prefers the current matched man or the proposing one in $O(1)$. Specifically, for a woman i, we store at *rank*$[i][j]$ the position of the man j in her preference list. This (lines 9-13), requires $O(n^2)$ preprocessing. In lines $14 - 31$ we perform the proposals.

See that if a woman receives a proposal, then she will either accept it or stay with her current matching. This leads to the following lemma.

Lemma 2.3.3. *After a woman receives a match, she never becomes unmatched again.*

Note that it is not necessary to check whether $ptr[m]$ becomes greater than $n-1$, which would mean that we popped a man who is unmatched, but all the women are already matched.

Lemma 2.3.4. *Algorithm 2.6 always halts and produces a marriage.*

Proof. Assume that a man gets popped in line 15, which means that he is unmatched, but has already proposed to all women. Then every woman has rejected him for some other man and, by Lemma 2.3.3, all women are matched. This is a contradiction, because this would mean that a man appears in more than one match, so necessarily an unmatched woman exists. Consequently, every time we pop a man from the queue, there will always exist a woman who is unmatched. Since each man proposes to each woman at most once, there exist at most n^2 proposals, so the algorithm halts. Finally, when the algorithm halts, no man is unmatched, hence the output is a marriage. □

Now are going to prove the output of the algorithm is stable.

Lemma 2.3.5. *Algorithm 2.6 produces a stable marriage.*

Proof. By Lemma 2.3.4, we know that it produces a marriage. Assume, to the contrary, that the output is not stable, which means that a blocking pair (m_i, w_j) exists. Because (m_i, w_j) does not belong to the matching, two cases are possible.

In the first case, the man m_i proposed to w_j and she rejected him (immediately or accepted him temporarily and later left him for a better proposal). This means that w_j prefers the man from the matching. A contradiction.

In the second case, the man m_i never proposed to w_j, which means that he matched with a woman who is higher on his preference list. Also a contradiction. □

As we perform at most $O(n^2)$ proposals and we can decide whether to accept a proposal or not in $O(1)$ (using $O(n^2)$ preprocessing), the total complexity is $O(n^2)$.

Theorem 2.3.6. *Algorithm 2.6 computes a stable marriage in $O(n^2)$.*

Definition 2.3.7. *Algorithm 2.6 operates as a men-proposing algorithm since a man first proposes to his most preferred woman, leaving the woman to accept the best proposal available to her.*

Theorem 2.3.8. *The men-proposing Algorithm 2.6 is man-optimal, as every man is matched to the best partner he could possibly obtain in any stable marriage.*

Proof. Suppose, for the sake of contradiction, that it is not man-optimal, and that the output of Algorithm 2.6 is a marriage M. This means that there exist some rejections in our algorithm such that the corresponding pairs are matched in another stable marriage M'. From those rejections, fix the first one. That is, a man m_i got rejected by woman w_j because w_j prefers m_z to m_i, and another stable marriage M' exists, where m_i is matched with w_j, i.e., $\{m_i, w_j\} \in M'$, and m_z is matched with w_k, i.e., $\{m_z, w_k\} \in M'$. If m_z prefers w_j to w_k, then $\{m_z, w_j\}$ is a blocking pair in M', which leads to a contradiction. If m_z prefers w_k to w_j, then it would be impossible for the first rejection to be that of m_i by w_j, because it would mean that w_j is already matched with m_z, implying that m_z was rejected by w_k in some previous iteration. This also leads to a contradiction. □

Although the output of the algorithm is man-optimal, it is the worst matching for women.

Theorem 2.3.9. *The men-proposing Algorithm 2.6 is female-pessimal, meaning every woman is matched with the least preferred valid partner she could have in any stable matching.*

Proof. Similar to the previous proof, assume to the contrary that woman w_j is matched with man m_i in the marriage output M of Algorithm 2.6 and that there exists another stable marriage $M^{'}$ such that w_j is matched with a man m_z whom she prefers less than m_i and m_i is matched with a woman w_k. Because M is man-optimal by Theorem 2.3.8, m_i prefers w_j to w_k. The pair $\{m_i, w_j\}$ is a blocking pair for $M^{'}$. This is a contradiction, since $M^{'}$ is stable. □

Further Variants of the Model The Gale and Shapley algorithm focuses on a one-to-one matching model, which means that each element from one set can be matched to exactly one element from the other set. Also, the sizes of the sets and the preference lists are the same. These assumptions are actually rare. In real-life scenarios, we may have many-to-one or even many-to-many assignments, incomplete lists, and preferences with ties. Additionally, stability is just one of many criteria used to evaluate the quality of a matching. In a later chapter, we will study some of these extended scenarios.

Chapter 3

Polyhedral Approaches to Matching Problems

3.1 Preliminaries

There exist many excellent references on linear programming and polyhedral combinatorics. This section is based mainly on [MG07] and [Sch03].

Let C be any subset of $\mathbb{R}^n$. We say that C is *convex* if for every two points $u, v \in C$ and for every $\lambda \in [0, 1]$, $u + \lambda d$ belongs to C, where $d = v - u$. In other words, we say that C is convex if for every $u, v \in C$, it also contains the line segment between u and v. It is easy to see that the intersection of convex sets is convex.

Let $S \subset \mathbb{R}^n$ be a set. The *convex hull* of S, denoted by *conv*(S), is the intersection of all the convex sets containing S. Expressed differently, it is the smallest convex set containing S. It is often more useful to describe the convex hull in terms of convex combinations.

Let $v_1, \ldots, v_k \in \mathbb{R}^n$ and $\lambda_1, \ldots, \lambda_k$ be non-negative reals such that $\lambda_1 + \ldots + \lambda_k = 1$. Then, $\lambda_1 v_1 + \ldots + \lambda_k v_k$ is called a *convex combination* of $v_1, \ldots, v_k$. Geometrically, convex combinations can be viewed as taking the line segments between the given points (by setting all coefficients to zero except those corresponding to two points) and then generating all points contained in the resulting shape as more coefficients are allowed to be non-zero. For two points, this generates the entire line segment connecting them, and for three *non-collinear* points, it generates the whole triangle (including its boundary). The set of all such points forms a convex set, called the convex hull of the given set. Therefore, $\text{conv}(S) = \{\sum_{i=1}^{k} \lambda_i v_i : k \in \mathbb{N}, v_i \in S, \lambda_i \geqslant 0, \sum_{i=1}^{k} \lambda_i = 1\}$, which is equivalent to the previous definition.

A set $S \subseteq \mathbb{R}^n$ is an *affine subspace* of $\mathbb{R}^n$ if $S = p + V = \{p + v : v \in V\}$ for some $p \in \mathbb{R}^n$ and some linear subspace V of $\mathbb{R}^n$. In other words, it is a linear subspace that has been shifted by a fixed vector. If $p \in V$, then the subspace is simply a linear subspace.

Recall the definition of a convex combination, but let $\lambda_i \in \mathbb{R}$ instead of $\lambda_i \in [0, 1]$. For two points, the result is the line passing through these two points. This line is formed by the affine combinations of these points (and it does not need to pass through the origin). In general, given $v_1, \ldots, v_k \in \mathbb{R}^n$ and $\lambda_1, \ldots, \lambda_k \in \mathbb{R}$ such that $\lambda_1 + \ldots + \lambda_k = 1$, the sum $\lambda_1 v_1 + \ldots + \lambda_k v_k$ is called an *affine combination* of $v_1, \ldots, v_k$.

The dimension of an affine subspace S, denoted by $\dim(S)$, is defined to be $\dim(V)$. Using affine combinations, we can generate a line, a plane, or a *hyperplane*, which is an affine subspace of dimension $n - 1$. Similarly to the definition of a convex hull, given a set $S \subseteq \mathbb{R}^n$, the *affine hull* of S ($\text{aff}(S)$) is the intersection of all affine subspaces containing S, or equivalently, the set of all possible affine combinations of a finite number of elements of

S.

We are also interested in knowing which points contribute to "extending" the affine hull (i.e., increasing its dimension). For example, in $\mathbb{R}^2$, the points $(1, 0)$, $(0, 1)$, and $(1, 1)$ are affinely independent because the first two define a line, and by adding $(1, 1)$, we extend the affine hull to all of $\mathbb{R}^2$. We say that the points $v_1, \ldots, v_k \in \mathbb{R}^n$ are *affinely independent* if them can be written as an affine combination of the remaining points. It can be proved that $v_1, \ldots, v_k \in \mathbb{R}^n$ are affinely independent if and only if the vectors $v_2 - v_1, \ldots, v_k - v_1$ are linearly independent.

A hyperplane in $\mathbb{R}^n$ describes all the solutions $x \in \mathbb{R}^n$ of a single linear equation of the form $a_1x_1 + a_2x_2 + \ldots + a_nx_n = b$ where at least one a_i is not zero and $b \in \mathbb{R}$. We can also express it using two inequalities $a_1x_1 + a_2x_2 + \ldots + a_nx_n \leqslant b$ and $a_1x_1 + a_2x_2 + \ldots + a_nx_n \geqslant b$. These inequalities are called *half-spaces* (and more specifically *closed* half-spaces because they contain their boundary).

A *polyhedron* $P \subseteq \mathbb{R}^n$ is the intersection of a finite number of halfspaces. This means that we could write $P = \{x \in \mathbb{R}^n : Ax \leqslant b\}$ where $A \in \mathbb{R}^{m \times n}$ and $b \in \mathbb{R}^m$. It is easy to see that polyhedra is convex because halfspaces are convex. We call z an *extreme point* (or *vertex*) of P if $z \in P$ and there do not exist distinct $x, y \in P$ and $\lambda \in (0, 1)$ such that $z = (1 - \lambda)x + \lambda y$ (which means that z is not an internal point of any segment from x to y such that $x, y \in P$). The *dimension* of a polyhedron P, denoted by $\dim(P)$, is the minimum dimension of an affine subspace containing P.

Let $P = \{x \in \mathbb{R}^n : Ax \leqslant b\}$ and denote a_i^T to be the i-th row of A. For a point $x^* \in P$, we say that the inequality $a_i^T x \leqslant b_i$ is *active* at x^* if $a_i^T x^* = b_i$. Let $A^= x \leqslant b^=$ be the subsystem of $Ax \leqslant b$ active at x^*. A point $x^* \in P$ is an extreme point of P if and only if the active constraint vectors at x^* contain n linearly independent rows, i.e., $\operatorname{rank}(A^=) = n$. In general, if a polyhedron is described by both equalities and inequalities, $P = \{x \in \mathbb{R}^n : Dx = d,\ Ax \leqslant b\}$, then $x^* \in P$ is an extreme point of P if and only if $\operatorname{rank}\begin{bmatrix} D \\ A^= \end{bmatrix} = n$.

An inequality $a^T x \leqslant b$ is called *valid* for P if it is satisfied by every point of P, that is, if $a^T x \leqslant b$ for all $x \in P$. The set $F = \{x \in P : a^T x = b\}$ is called the *face* of P induced by this valid inequality. A face F of P is called a *facet* if $\dim(F) = \dim(P) - 1$. In this case, we say that the inequality $a^T x \leqslant b$ is *facet-defining* for P. Equivalently, if $d = \dim(P)$ and the inequality is not satisfied with equality by every point of P, then it is facet-defining if and only if its equality set contains d affinely independent points of P.

A polyhedron $P \subseteq \mathbb{R}^n$ is called a *polytope* if it is bounded (in other words there exists $M > 0$ such that $\|x\| < M$, $\forall x \in P$, where $\|\|$ is the euclidean norm). A polytope has a finite number of extreme points and every other point of that polytope can be expressed as a convex combination of its extreme points. In fact, if C is the set of the extreme points of P, we can describe it with the convex hull of C, which means that $P = \operatorname{conv}(C)$. Hence, there are two representations of a polytope P. The first one is by defining the linear inequalities $Ax \leqslant b$, which is called the *H-description* (or *linear description*) and the second one by its vertices, which is called the *V-description*.

A polyhedron $P \subseteq \mathbb{R}^n$ with at least one vertex is called *integral* if all of its vertices are integral. In particular, a polytope is integral if and only if $P = \operatorname{conv}(P \cap \mathbb{Z}^n)$.

Let $c : \{1, \ldots, n\} \to \mathbb{R}$. In linear programming (LP) we are interested to maximize a function $c^T x$ (which is often called an *objective function*) over a polyhedron. This is a linear optimization problem, which we call it a *primal problem.* The *constraints* of that problem are the linear description of it. In this thesis, we will mostly use maximization problems with nonnegative variables. Unless stated otherwise, the variables of a linear program are assumed to be real-valued. Hence, the primal problem can be expressed as

$$\begin{aligned} \max\ & c^T x \\ \text{s.t. } & Ax \leqslant b \\ & x \geqslant 0 \end{aligned} \tag{3.1}$$

Any vector $x \in \mathbb{R}^n$ satisfying all constraints is called a *feasible solution*. The set of all feasible solutions is called the *feasible region* and in the case of a linear program, this region is a polyhedron. Any feasible solution that achieves the maximum possible value of the objective function is called an *optimal solution*. There can be multiple optimal solutions, or none at all. If the objective function can attain arbitrarily large values, we say that the LP is *unbounded*. Such a problem can be solved using known algorithms that are polynomial in the input size, for example, the ellipsoid method.

We also recall the basic notion of linear programming duality. Consider the primal linear program of 3.1 where $A \in \mathbb{R}^{m\times n}$, $b \in \mathbb{R}^m$ and $c \in \mathbb{R}^n$. The corresponding *dual problem* is

$$\begin{aligned} \min \quad & b^T y \\ \text{s.t.} \quad & A^T y \geqslant c, \\ & y \geqslant 0. \end{aligned}$$

The variables x are called the *primal variables*, while the variables y are called the *dual variables*. A vector satisfying the constraints of the primal problem is called a *primal feasible solution*, and a vector satisfying the constraints of the dual problem is called a *dual feasible solution*.

The weak duality theorem states that, for every primal feasible solution x and every dual feasible solution y, we have

$$c^T x \leqslant b^T y.$$

Therefore, the value of any dual feasible solution gives an upper bound on the value of any primal feasible solution.

The strong duality theorem states that, if the primal problem has an optimal solution and its optimal value is finite, then the dual problem also has an optimal solution and the two optimal values are equal.

We recall two standard facts relating linear programs and vertices of polyhedra.

Proposition 3.1.1. *Let $P \subseteq \mathbb{R}^n$ be a polyhedron, and let $v \in P$. If v is a vertex of P, then there exists a nonzero vector $c \in \mathbb{R}^n$ such that*

$$c^T v > c^T y$$

for every $y \in P\backslash\{v\}$.

Theorem 3.1.2. *(Fundamental theorem of linear programming) Let $P \subseteq \mathbb{R}^n$ be a polyhedron. If the linear program*

$$\max\{c^T x : x \in P\}$$

has an optimal solution and P has at least one vertex, then it has an optimal solution that is a vertex of P.

Consider a linear program $\max\{c^T x : Ax = b,\ x \geqslant 0\}$ (which is often called *standard form*), where $A \in \mathbb{R}^{m\times n}$ has rank m. A set $B \subseteq \{1,\dots,n\}$ with $|B| = m$ is called a *basis* if the submatrix A_B consisting of the columns of A indexed by B is nonsingular. The variables indexed by B are called *basic variables*, while the remaining variables are called *nonbasic variables*. Setting the nonbasic variables equal to 0 and solving $A_B x_B = b$ gives the associated *basic solution*, namely

$$x_B = A_B^{-1} b, \qquad x_N = 0.$$

If this solution also satisfies $x_B \geqslant 0$, then it is called a *basic feasible solution*.

For linear programs in standard form, the basic feasible solutions are exactly the extreme points of the feasible polyhedron. Therefore, if such a linear program has an optimal solution, then it has an optimal basic feasible solution.

If we consider the LP problem but restrict $x \in \mathbb{Z}_+^n$, it becomes an *integer program* (IP). Unlike LP, this problem is difficult, i.e., it is known to be NP-hard. However, if the feasible region of the LP relaxation is integral, then we can solve the LP instead of the IP and obtain an integer optimal solution. The problem is still difficult if the size of the linear description is exponential in n or m. One way to address this issue is by solving the *separation* problem, in which, for a given point $x \in \mathbb{R}_+^n$, we decide whether it violates any of the inequalities of $Ax \leqslant b$ and return one such inequality.

A matrix A is *totally unimodular* if every square submatrix of A has determinant equal to 0, 1 or -1. We will prove the following lemma.

Lemma 3.1.3. *Let A be a totally unimodular matrix and let A' be the matrix obtained from A by appending a unit vector e_i as a new column. Then A' is totally unimodular.*

Proof. Let B be an arbitrary square submatrix of A'. If B does not contain the appended column e_i, then B is a square submatrix of A, and therefore $\det(B) \in \{-1, 0, 1\}$.

Suppose now that B contains the appended column. The restriction of e_i to the rows of B is either the zero vector or a unit vector. In the first case, B has a zero column, so $\det(B) = 0$. In the second case, expanding along this column gives

$$\det(B) = \pm\det(C),$$

where C is a square submatrix of A. Since A is totally unimodular, $\det(C) \in \{-1, 0, 1\}$, and hence $\det(B) \in \{-1, 0, 1\}$.

Thus every square submatrix of A' has determinant in $\{-1, 0, 1\}$, so A' is totally unimodular. □

An *LP relaxation* of an IP is obtained by removing the integrality constraints on the variables and allowing them to take arbitrary real values within the feasible region. We now show how total unimodularity can be used to solve IP via LP relaxation, following the approach of [MG07, p. 145].

Lemma 3.1.4. *Let $\max\{c^T x : Ax \leqslant b,\ x \geqslant 0\}$ be the LP relaxation of an IP, where A is totally unimodular and $b \in \mathbb{Z}^m$. Suppose that the LP has an optimal solution. Then it also admits an integral optimal solution.*

Proof. Introduce slack variables $s \in \mathbb{R}^m$ and $s \geqslant 0$, so that $Ax + s = b$. Then the LP can be written in standard form as

$$\max\left\{ \begin{bmatrix} c \\ 0 \end{bmatrix}^T \begin{bmatrix} x \\ s \end{bmatrix} : \begin{bmatrix} A & I_m \end{bmatrix} \begin{bmatrix} x \\ s \end{bmatrix} = b,\ x \geqslant 0,\ s \geqslant 0 \right\}.$$

Let

$$M = \begin{bmatrix} A & I_m \end{bmatrix}.$$

Since A is totally unimodular, and I_m is obtained by appending the unit vectors $e_1, \ldots, e_m$, the previous result implies that $M = [A \ I_m]$ is also totally unimodular.

By the fundamental theorem of linear programming, since the linear program has an optimal solution, it has an optimal basic feasible solution (we could get one using the simplex method). Let

$$z^* = \begin{bmatrix} x^* \\ s^* \end{bmatrix}$$

be such an optimal basic feasible solution. Then there exists a basis matrix M_B, consisting of linearly independent columns of M, such that

$$z_B^* = M_B^{-1} b,$$

and all nonbasic variables are equal to 0.

Now M_B is a nonsingular square submatrix of M. Since M is totally unimodular,

$$\det(M_B) \in \{-1, 0, 1\}.$$

But M_B is nonsingular, so

$$\det(M_B) = \pm 1.$$

Therefore,

$$M_B^{-1} = \frac{\operatorname{adj}(M_B)}{\det(M_B)}$$

has integer entries, because M_B has integer entries and $\det(M_B) = \pm 1$. Since $b \in \mathbb{Z}^m$, it follows that

$$z_B^* = M_B^{-1} b$$

is integral. The nonbasic variables are zero, hence they are also integral. Therefore,

$$z^* = \begin{bmatrix} x^* \\ s^* \end{bmatrix} \in \mathbb{Z}^{n+m}.$$

In particular,

$$x^* \in \mathbb{Z}^n.$$

Thus the original linear program has an integral optimal solution. □

Consequently, we can deduce the following corollary.

Corollary 3.1.5. *The feasible region of a linear program with totally unimodular constraint matrix and integral right-hand size forms an integral polyhedron.*

To see why, suppose for contradiction, that P has a non-integral vertex v. Since v is a vertex, there exists a vector c such that v is the unique optimal solution of $\max\{c^T x : x \in P\}$. Since the constraint matrix is totally unimodular and the right-hand side is integral, the above linear program admits an integral optimal solution x^*. However, the optimal solution is unique, and therefore $x^* = v$. This implies that v is integral, which contradicts our assumption. Hence every vertex of P is integral, and so P is integral.

3.2 The Bipartite Matching Polytope

We solved the maximum bipartite matching problem algorithmically in 2.2. In this section, we will solve the same problem using linear programming terminology. Let $G = (V_1 \cup V_2, E)$ be a bipartite graph with n vertices $v_1, \dots, v_n$ and m edges $e_1, \dots, e_m$ and let $\delta(v)$ denote the set of edges incident to v. The IP describing the problem is the following:

$$\begin{aligned} \max \quad & \sum_{e \in E} x_e \\ \text{s.t.} \quad & \sum_{e \in \delta(v)} x_e \leqslant 1, \quad \forall v \in V(G), \\ & x_e \geqslant 0, \quad \forall e \in E, \\ & x_e \in \mathbb{Z}, \quad \forall e \in E. \end{aligned} \tag{3.2}$$

or in compact form:

$$\begin{aligned} \max \quad & \sum_{j=1}^{m} x_j \\ \text{s.t.} \quad & Ax \leqslant \mathbf{1}, \\ & x \geqslant 0, \\ & x \in \mathbb{Z}^m. \end{aligned} \tag{3.3}$$

where $\mathbf{1}$ is a vector with n components, each equal to 1 and the matrix A is defined as:

$$A_{v,e} = \begin{cases} 1, & \text{if } v \text{ is incident to edge } e, \\ 0, & \text{otherwise.} \end{cases}$$

In fact, x's components can be 0 or 1 based on the constraint matrix, so we could simply write $x_j \in \{0, 1\}$. Consider now the LP relaxation of 3.4.

$$\begin{aligned} \max \quad & \sum_{j=1}^{m} x_j \\ \text{s.t.} \quad & Ax \leqslant \mathbf{1}, \\ & x \geqslant 0. \end{aligned} \tag{3.4}$$

Since the feasible region is non-empty and bounded, it is a polytope, guaranteeing that the LP relaxation has an optimal solution at a vertex. The polytope of **??** is integral. To prove it, it's sufficient (see 3.1.5) to show that A is totally unimodular, since the right-hand side vector $\mathbf{1}$ is integral.

Lemma 3.2.1. *Let $G = (V_1 \cup V_2, E)$ be a bipartite graph, and let A be its vertex-edge incidence matrix. Then A is totally unimodular.*

Proof. We prove that every square submatrix of A has determinant equal to 0, 1, or -1.

Let Q be an arbitrary square submatrix of A. We use induction on the size of Q. If Q is a 1×1 matrix, then its determinant is either 0 or 1, since all entries of A are 0 or 1.

Now suppose that Q has size $k \times k$, with $k \geqslant 2$, and assume that the claim holds for all smaller square submatrices of A.

Since A is the vertex-edge incidence matrix of G, every column of A contains exactly two entries equal to 1, one corresponding to each endpoint of the edge. Therefore, every column of Q contains at most two entries equal to 1.

If some column of Q contains no entry equal to 1, then this column is zero, and hence

$$\det(Q) = 0.$$

If some column of Q contains exactly one entry equal to 1, then we expand the determinant along this column. We get

$$\det(Q) = \pm \det(Q'),$$

where Q' is a square submatrix of A of size $(k-1) \times (k-1)$. By the induction hypothesis,

$$\det(Q') \in \{-1, 0, 1\}.$$

Therefore,

$$\det(Q) \in \{-1, 0, 1\}.$$

It remains to consider the case where every column of Q contains exactly two entries equal to 1. Since G is bipartite, every edge has one endpoint in V_1 and one endpoint in V_2. Hence, in every column of Q, one of the two 1's appears in a row corresponding to a vertex of V_1, and the other appears in a row corresponding to a vertex of V_2.

Therefore, the sum of the rows of Q corresponding to vertices in V_1 is equal to the sum of the rows of Q corresponding to vertices in V_2. Hence, the rows of Q are linearly dependent, and so

$$\det(Q) = 0.$$

In all cases, we have

$$\det(Q) \in \{-1, 0, 1\}.$$

□

Since the polytope of the LP relaxation contains all feasible solutions of the IP, the LP relaxation can only have an optimal value at least as large as that of the IP. However, the polytope of the LP relaxation is integral and since it has an optimal solution at a vertex, this optimal solution is also integral. This optimal solution is feasible for the original IP as well, since each component of the solution is either 0 or 1. Consequently, the LP relaxation and the IP have the same optimal value, so we can solve the LP instead of the IP to obtain a maximum matching.

Another famous problem in graph theory, is the *minimum vertex cover*. A vertex cover of graph G is a minimum set of nodes such that each edge of the graph has at least one endpoint in the set. This problem is NP-hard in a general graph. However, if the graph is bipartite, we can solve it in polynomial time. In fact, the following theorem holds.

Theorem (König's Theorem)**.** *Let $G = (V_1 \cup V_2, E)$ be a bipartite graph. Then the maximum cardinality of a matching in G is equal to the minimum cardinality of a vertex cover in G.*

Proof. Let A be the vertex-edge incidence matrix of G. Consider the LP relaxation 3.4 of the maximum matching problem. We have shown that, since A is totally unimodular and the right-hand side vector is integral, the optimal value of this relaxation is equal to the maximum cardinality of a matching in G.

The dual of this LP relaxation is

$$\begin{aligned} \min \quad & \mathbf{1}^T y \\ \text{s.t.} \quad & A^T y \geqslant \mathbf{1}, \\ & y \geqslant 0. \end{aligned}$$

This is exactly the LP relaxation of the minimum vertex cover problem. Indeed, the IP for minimum vertex cover is

$$\begin{aligned} \min \quad & \mathbf{1}^T y \\ \text{s.t.} \quad & A^T y \geqslant \mathbf{1}, \\ & \quad\; y \geqslant 0, \\ & \quad\; y \in \mathbb{Z}^n. \end{aligned}$$

Here $y_v = 1$ means that the vertex v is chosen in the vertex cover. The constraint $A^T y \geqslant \mathbf{1}$ says that for every edge, at least one of its endpoints must be chosen.

Since A is totally unimodular, it is easy to see that A^T is also totally unimodular. Therefore, again by the total unimodularity result, the LP relaxation of the minimum vertex cover problem has an integral optimal solution. Moreover, in an optimal integral solution we may assume that every component is either 0 or 1, since if $y_v > 1$, replacing y_v by 1 keeps all constraints feasible and does not increase the objective value. Hence, the optimal value of the vertex cover LP relaxation is equal to the minimum cardinality of a vertex cover in G.

Finally, by strong duality, the optimal values of the two LP relaxations are equal. The first one is equal to the maximum cardinality of a matching in G, and the second one is equal to the minimum cardinality of a vertex cover in G. Therefore, these two quantities are equal. $\square$

3.3 The Stable Matching Polytope

In Section 2.3, we studied the stable marriage problem from an algorithmic point of view. In particular, the Gale–Shapley algorithm computes a stable marriage in polynomial time [GS62]. In this section, we study the same problem from a polyhedral point of view.

We keep the same assumptions as before. There are n men and n women, all preference lists are complete and strict, and every feasible solution is a marriage. Let

$$U = \{m_1, m_2, \ldots, m_n\}$$

be the set of men and

$$V = \{w_1, w_2, \ldots, w_n\}$$

be the set of women. For a man m, we write $w' >_m w$ if m prefers woman w' to woman w. Similarly, for a woman w, we write $m' >_w m$ if w prefers man m' to man m.

For every pair $(m, w) \in U \times V$, define a variable

$$x_{mw} = \begin{cases} 1, & \text{if } m \text{ is matched with } w, \\ 0, & \text{otherwise.} \end{cases}$$

Then the stable marriage problem can be written as the following IP:

$$\begin{aligned} \sum_{w \in V} x_{mw} &= 1, && \forall m \in U, \\ \sum_{m \in U} x_{mw} &= 1, && \forall w \in V, \\ x_{mw} + \sum_{\substack{w' \in V \\ w' >_m w}} x_{mw'} + \sum_{\substack{m' \in U \\ m' >_w m}} x_{m'w} &\geqslant 1, && \forall (m, w) \in U \times V, \\ x_{mw} &\in \{0, 1\}, && \forall (m, w) \in U \times V. \end{aligned} \tag{3.5}$$

The first two families of constraints say that each man is matched with exactly one woman and each woman is matched with exactly one man. Hence, they describe a marriage.

The third family of constraints expresses stability. Fix a pair (m, w). The inequality says that at least one of the following must happen:

$$m \text{ is matched with } w,$$

or

$$m \text{ is matched with a woman he prefers to } w,$$

or

$$w \text{ is matched with a man she prefers to } m.$$

If none of these happens, then m and w prefer each other to their assigned partners. Therefore, (m, w) is a blocking pair.

Lemma 3.3.1. *The feasible integer points of 3.5 are exactly the incidence vectors of stable marriages.*

Proof. Let x be a feasible integer point of 3.5. Since $x_{mw} \in \{0,1\}$, the first two families of constraints define a marriage. Let N be this marriage.

We show that N is stable. Suppose for the sake of contradiction that there exists a blocking pair (m, w). Then m is not matched with w, so

$$x_{mw} = 0.$$

Also, since m prefers w to his assigned partner, m is not matched with any woman w' such that $w' \succ_m w$. Thus,

$$\sum_{\substack{w' \in V \\ w' \succ_m w}} x_{mw'} = 0.$$

Similarly, since w prefers m to her assigned partner, w is not matched with any man m' such that $m' \succ_w m$. Hence,

$$\sum_{\substack{m' \in U \\ m' \succ_w m}} x_{m'w} = 0.$$

Therefore, the stability constraint for the pair (m, w) is violated. This is a contradiction. Hence, N is stable.

Conversely, let N be a stable marriage and let χ^N be its incidence vector. The first two families of constraints are clearly satisfied. Now fix a pair (m, w). If m is matched with w, then the stability inequality is satisfied because $x_{mw} = 1$. Otherwise, since N is stable, the pair (m, w) is not blocking. Therefore, either m is matched with a woman he prefers to w, or w is matched with a man she prefers to m. Hence, the stability inequality is again satisfied. □

We now define the polytope associated with stable marriages.

Definition 3.3.2. *The stable matching polytope, also called the stable marriage polytope in this setting, is*

$$P_{SM} = \operatorname{conv}\left\{\chi^N \in \mathbb{R}^{U \times V} : N \textit{ is a stable marriage}\right\}.$$

Thus, P_{SM} is the convex hull of all incidence vectors of stable marriages. It contains all stable marriages as integral points, but it may also contain fractional points. For example, if N_1 and N_2 are two different stable marriages, then $\frac{1}{2}\chi^{N_1} + \frac{1}{2}\chi^{N_2}$ is usually fractional and also belongs to P_{SM}.

Let Q_{SM} be the polyhedron obtained from 3.5 by replacing the constraints $x_{mw} \in \{0,1\}$ with $x_{mw} \geqslant 0$. Since the first two families of constraints imply $x_{mw} \leqslant 1$, no separate upper bound is needed.

The important question is whether this linear relaxation introduces fractional vertices. In fact, the relaxation is *exact*.

Theorem 3.3.3 (Vande Vate [Vat89]; Rothblum [Rot92])**.** *For every stable marriage instance with complete and strict preference lists,*

$$Q_{SM} = P_{SM}.$$

Equivalently, every extreme point of Q_{SM} is the incidence vector of a stable marriage.

The nontrivial part of the theorem is that the polyhedron Q_{SM} has no fractional extreme points. Therefore, every fractional feasible point of Q_{SM} is only a convex combination of stable marriages, and not a new vertex of the relaxation.

The nontrivial part of the theorem is that the polyhedron Q_{SM} has no fractional extreme points, so Q_{SM} is *integral*. Therefore, every fractional feasible point of Q_{SM} can be written as a convex combination of incidence vectors of stable marriages, and is not a new vertex of the relaxation.

This gives a complete linear description of the stable matching polytope:

$$P_{SM} = \left\{ x \in \mathbb{R}^{U \times V}_{\geqslant 0} : \begin{array}{ll} \sum_{w \in V} x_{mw} = 1, & \forall m \in U, \\ \sum_{m \in U} x_{mw} = 1, & \forall w \in V, \\ x_{mw} + \sum_{\substack{w' \in V \\ w' >_m w}} x_{mw'} + \sum_{\substack{m' \in U \\ m' >_w m}} x_{m'w} \geqslant 1, & \forall (m, w) \in U \times V. \end{array} \right\}.$$

The formulation has n^2 variables, $2n$ assignment constraints, and n^2 stability constraints, apart from non-negativity. Hence, it is a compact linear formulation.

Consequences. The Gale–Shapley algorithm is enough if we only want to find one stable marriage. The polyhedral formulation is useful when we want to optimize a linear objective over all stable marriages. For example, if each pair (m, w) has weight c_{mw}, we may solve

$$\begin{aligned} \max \quad & \sum_{m \in U} \sum_{w \in V} c_{mw} x_{mw} \\ \text{s.t.} \quad & x \in Q_{SM}. \end{aligned} \tag{3.6}$$

Because every extreme point of Q_{SM} is integral, there exists an optimal extreme point of 3.6 that is the incidence vector of a stable marriage. Therefore, the optimal stable marriage problem with a linear objective can be solved as LP.

This section gives only a brief polyhedral view of the stable marriage problem. We introduced the natural integer formulation, defined the stable matching polytope, and stated the main result that its linear relaxation is integral. Together with the maximum bipartite matching formulation studied in the previous section, this gives another example where the natural linear relaxation is integral. However, this is not the case for many other matching problems. In the next chapter, we will study one such case.

Chapter 4

Pareto Matchings in one-to-one Models

4.1 Model and Definitions

In Sections 2.3 and 3.3, we studied the stable marriage problem, where there are n men and n women, and each participant has a preference list containing all members of the opposite side. Therefore, every stable matching matches all participants.

In this chapter, we study a different setting, where the two sides need not have the same size, and only one side has preferences over the other side. For these reasons, it is natural to formulate this matching problem in the setting of a housing market.

The house allocation problem has been studied from an algorithmic point of view in [ACMM04]. An integer programming formulation for the same setting was later proposed by Eirinakis, Magos, and Mourtos [EMM]. In this chapter, we follow the notation and the basic definitions used in these works.

House allocation is a one-sided matching problem. We are given a finite set

$$A = \{a_1, \ldots, a_r\}$$

of agents and a finite set

$$H = \{h_1, \ldots, h_s\}$$

of houses. The houses are indivisible objects, and no monetary transfers are allowed. In contrast with two-sided matching models, only the agents have preferences.

Each agent $a \in A$ finds acceptable only some houses in H. We denote by $P(a)$ the preference list of agent a. The list $P(a)$ is strict, so no ties are allowed. If $h, h' \in P(a)$, then

$$h >_a h'$$

means that agent a prefers house h to house h'.

The acceptable pairs define a bipartite graph

$$G = (A, H, E),$$

where

$$E = \{(a, h) \in A \times H : h \in P(a)\}.$$

Thus, an edge (a, h) means that house h is acceptable to agent a.

For each house $h \in H$, let

$$Q(h) = \{a \in A : h \in P(a)\}.$$

Therefore, $Q(h)$ is the set of agents whose preference list contains house h.

A matching is a set $M \subseteq E$ such that each agent is assigned to at most one house and each house is assigned to at most one agent. If $(a, h) \in M$, then we write

$$M(a) = h \quad \text{and} \quad M(h) = a.$$

If an agent or a house is not assigned in M, then we write

$$M(z) = \varnothing, \quad z \in A \cup H.$$

Thus, in this model, every feasible allocation is *one-to-one*: no agent receives more than one house and no house is given to more than one agent.

A matching M' is a *Pareto improvement* over a matching M if every agent is at least as well off in M' as in M, and at least one agent is strictly better off. Here, being unmatched is considered worse than being assigned any acceptable house. A matching M is Pareto optimal if no Pareto improvement over M exists.

We next recall three properties of matchings that will be useful for describing Pareto optimality.

A matching M is *maximal* if there is no acceptable pair $(a, h) \in E$ such that

$$M(a) = M(h) = \varnothing.$$

In other words, no additional acceptable pair can be added to M without violating the matching constraints.

A matching M is *trade-in free* if there is no agent $a \in A$ and no house $h \in P(a)$ such that

$$M(a) \neq \varnothing, \qquad M(h) = \varnothing, \qquad \text{and} \qquad h >_a M(a).$$

Thus, no matched agent can improve her assignment by replacing her current house with an unmatched house.

A matching M is *coalition free* if it does not admit a coalition. A coalition is a sequence of matched agents

$$C = \langle a_{i_0}, a_{i_1}, \ldots, a_{i_{k-1}} \rangle, \quad k \geqslant 2,$$

such that

$$M(a_{i_{c+1}}) >_{a_{i_c}} M(a_{i_c}), \quad c = 0, \ldots, k-1,$$

where the indices are taken modulo k. If such a sequence exists, then the agents in C can cyclically exchange their assigned houses, and every agent in the sequence becomes strictly better off.

The following characterization connects these three properties with Pareto optimality.

Proposition 4.1.1 ([ACMM04, Chapter 2, p. 6]). *A matching M is Pareto optimal if and only if M is maximal, trade-in free, and coalition free.*

This characterization is the basis of the integer programming formulation that will be presented in the next section.

For this formulation, it is useful to define coalitions independently of a specific matching. A plausible coalition is a set of agent-house pairs

$$C = \{(a_{i_c}, h_{i_c}) : c = 0, \ldots, k-1\}, \quad k \geqslant 2,$$

such that, for every $c = 0, \ldots, k-1$,

$$h_{i_{c+1}} >_{a_{i_c}} h_{i_c}, \qquad \{h_{i_c}, h_{i_{c+1}}\} \subseteq P(a_{i_c}),$$

where again the indices are taken modulo k. If all pairs of such a set appear simultaneously in a matching, then the corresponding agents form a coalition. We denote by $\mathcal{C}$ the family of all plausible coalitions.

Finally, for every acceptable pair $(a, h) \in E$, we define a binary variable x_{ah}. For a matching M, its incidence vector is the vector $x^M \in \{0,1\}^E$ given by

$$x^M_{ah} = \begin{cases} 1, & \text{if } (a,h) \in M, \\ 0, & \text{otherwise.} \end{cases}$$

These incidence vectors will later be used to define the Pareto matching polytope.

4.2 An Integer Programming Formulation

4.3 The Linear Relaxation and Fractional Solutions

4.4 The Pareto Matching Polytope

4.5 Computational Examples and Candidate Cutting Inequalities

References

[ACMM04] David J Abraham, Katarína Cechlárová, David F Manlove, and Kurt Mehlhorn. Pareto optimality in house allocation problems. In *Algorithms and Computation: 15th International Symposium, ISAAC 2004, Proceedings*, pages 3–15. Springer, 2004.

[BBST24] Aaron Bernstein, Joakim Blikstad, Thatchaphol Saranurak, and Ta-Wei Tu. Maximum flow by augmenting paths in $n^{2+o(1)}$ time. In *2024 IEEE 65th Annual Symposium on Foundations of Computer Science (FOCS)*, pages 2056–2077, 2024.

[BCK+23] Jan Van Den Brand, Li Chen, Rasmus Kyng, Yang P. Liu, Richard Peng, Maximilian Probst Gutenberg, Sushant Sachdeva, and Aaron Sidford. A deterministic almost-linear time algorithm for minimum-cost flow. In *2023 IEEE 64th Annual Symposium on Foundations of Computer Science (FOCS)*, pages 503–514, 2023.

[Ber57] Claude Berge. Two theorems in graph theory. *Proceedings of the National Academy of Sciences*, 43(9):842–844, 1957.

[CK24] Julia Chuzhoy and Sanjeev Khanna. Maximum bipartite matching in ⊠2+⊠(1) time via a combinatorial algorithm. In *Proceedings of the 56th Annual ACM Symposium on Theory of Computing*, page 83–94. Association for Computing Machinery, 2024.

[Din70] Yefim A. Dinitz. Algorithm for solution of a problem of maximum flow in networks with power estimation. *Soviet Mathematics Doklady*, 11:1277–1280, 1970.

[Din06] Yefim Dinitz. Dinitz' algorithm: The original version and even's version. In Oded Goldreich, Arnold L. Rosenberg, and Alan L. Selman, editors, *Theoretical Computer Science: Essays in Memory of Shimon Even*, pages 218–240. Springer, Berlin, Heidelberg, 2006.

[EMM] P Eirinakis, D Magos, and I Mourtos. A model for house allocation. *2015 SCinTE*, page 19.

[GS62] D. Gale and L. S. Shapley. College admissions and the stability of marriage. *The American Mathematical Monthly*, 69(1):9–15, 1962.

[HK73] John E Hopcroft and Richard M Karp. An n^5/2 algorithm for maximum matchings in bipartite graphs. *SIAM Journal on computing*, 2(4):225–231, 1973.

[MG07] Jiří Matoušek and Bernd Gärtner. *Understanding and Using Linear Programming*. Universitext. Springer Science & Business Media, 2007.

[Pet91] Julius Petersen. Die theorie der regulären graphs, 1891.

[Rot92] Uri G. Rothblum. Characterization of stable matchings as extreme points of a polytope. *Mathematical Programming*, 54(1–3):57–67, 1992.

[Sch03] Alexander Schrijver. *Combinatorial Optimization: Polyhedra and Efficiency, Volume A*, volume 24 of *Algorithms and Combinatorics*. Springer-Verlag, Berlin, Heidelberg, 2003.

[Vat89] John H. Vande Vate. Linear programming brings marital bliss. *Operations Research Letters*, 8(3):147–153, 1989.